

Qwen3-VL 2B: Small Model, MASSIVE Power! Run Anywhere with a Smile



Fahd Mirza
48.1K subscribers

Join

Subscribe

 31



 Share

 Ask

596 views Oct 21, 2025 #qwen3vl #qwen3vl2b
This video locally installs Qwen3-VL-2B — the most powerful vision-language model in the Qwen series to date.

 **Hugging Face**

Search models, datasets,

Models

Datasets

Spaces

Docs

Pricing

Log In

Sign Up

 **Qwen/Qwen3-VL-2B-Instruct**

like 8 Follow Qwen 54.4k

Image-Text-to-Text Transformers Safetensors qwen3_vl image-to-text conversational arxiv:4 papers License: apache-2.0

Model card Files xet Community

Train Deploy Use this model

 Qwen Chat

Qwen3-VL-2B-Instruct

Meet Qwen3-VL — the most powerful vision-language model in the Qwen series to date.

This generation delivers comprehensive upgrades across the board: superior text understanding & generation, deeper visual perception & reasoning, extended context length, enhanced spatial and video dynamics comprehension, and stronger agent interaction capabilities.

Downloads last month

8



 Safetensors

Model size 2B params Tensor type BF16

 Chat template  Files info

 Inference Providers **NEW**

 Image-Text-to-Text

This model isn't deployed by any Inference Provider.  Ask for provider support



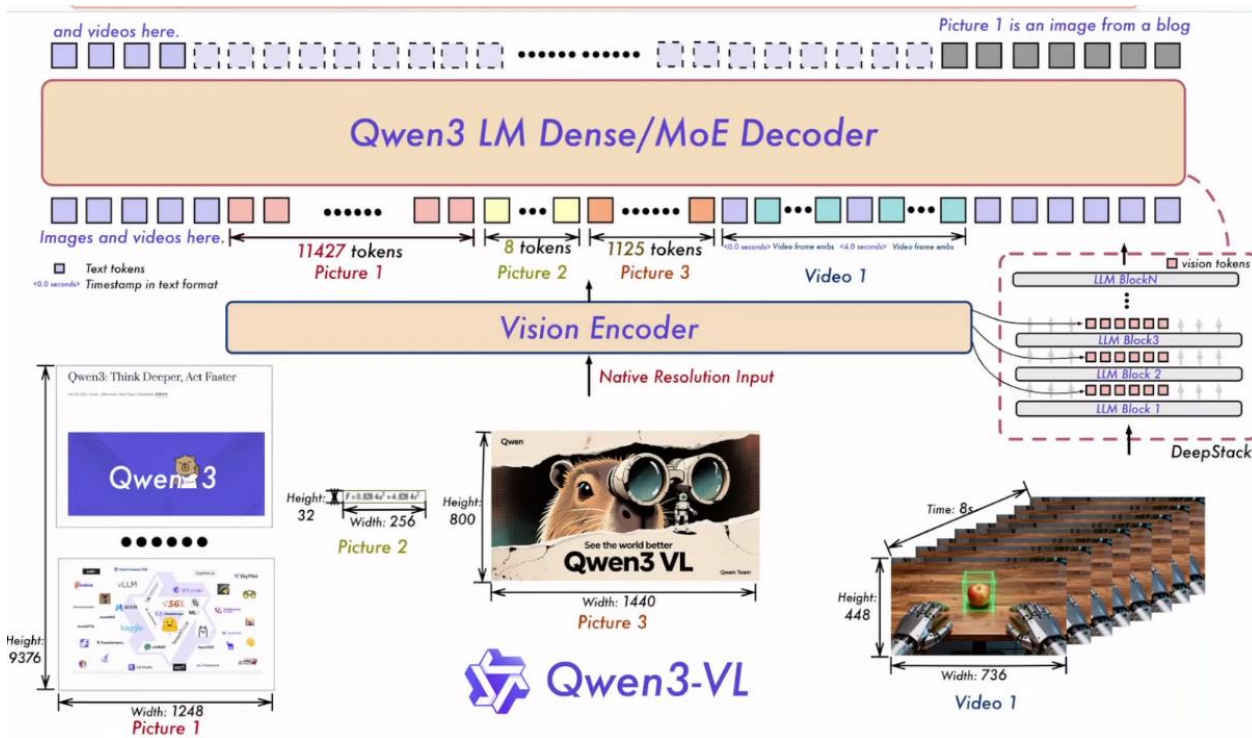
QWEN3-VL PARTY
Images x Videos x Text
11:07

Qwen3-VL-4B: Small Local AI Vision Model We All Have Been Waiting For
Fahd Mirza • 5.4K views • 6 days ago
This video locally installs Qwen3-VL-4B Instruct— the most powerful small vision-language model in the Qwen series to date. This video is sponsored by <https://www.eigent.ai>, the World's...



MIND BLOWING QWEN3-VL 30B
10:11

Qwen3-VL — The Most Powerful Vision-Language Model: Install and Test
Fahd Mirza • 3.6K views • 2 weeks ago
This video locally installs Qwen3-VL — the most powerful vision-language model in the Qwen series to date. Buy Me a Coffee to support the channel: <https://ko-fi.com/fahdmirza> #qwen3vl...



```
(ai) Ubuntu@0127-dsm2-ty6k-prxm70026:~$ conda install -c conda-forge --override-channels notebook -y
conda install -c conda-forge --override-channels ipywidgets -y
jupyter notebook
Channels:
- conda-forge
Platform: linux-64
Collecting package metadata (repodata.json): \
```

```
localhost:8888/notebooks/Untitled11.ipynb 150% ☆
jupyter Untitled11 Last Checkpoint: 2 minutes ago
File Edit View Run Kernel Settings Help
+ ✂ 📄 📁 ▶ ⏮ ⏭ ⏪ ⏩ Code ▾ JupyterLab 🗑 ⚙ Pyt

[+]: import os
os.environ['HF_HUB_DISABLE_PROGRESS_BARS'] = '1'

import torch
from transformers import Qwen3VLForConditionalGeneration, AutoProcessor

model = Qwen3VLForConditionalGeneration.from_pretrained(
    "Qwen/Qwen3-VL-2B-Instruct",
    torch_dtype=torch.bfloat16,
    attn_implementation="flash_attention_2",
    device_map="auto",
)
processor = AutoProcessor.from_pretrained("Qwen/Qwen3-VL-2B-Instruct")

[ ]:
```

```
jupyter Untitled11 Last Checkpoint: 8 minutes ago
File Edit View Run Kernel Settings Help

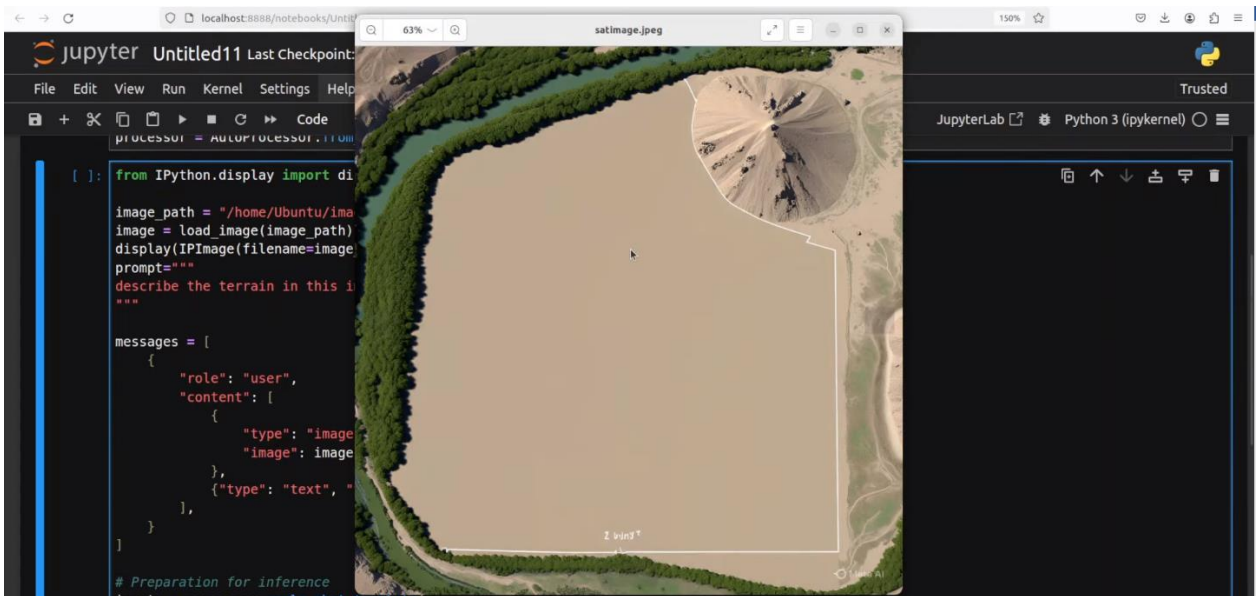
processor = AutoProcessor.from_pretrained( Qwen/Qwen2-VL-72B-Instruct )

[ ]: from IPython.display import display, Image as IPImage

image_path = "/home/ubuntu/images/planner/satimage.jpeg"
image = load_image(image_path)
display(IPImage(filename=image_path))
prompt="""
describe the terrain in this image.
"""

messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "image",
                "image": image,
            },
            {"type": "text", "text": prompt},
        ],
    }
]

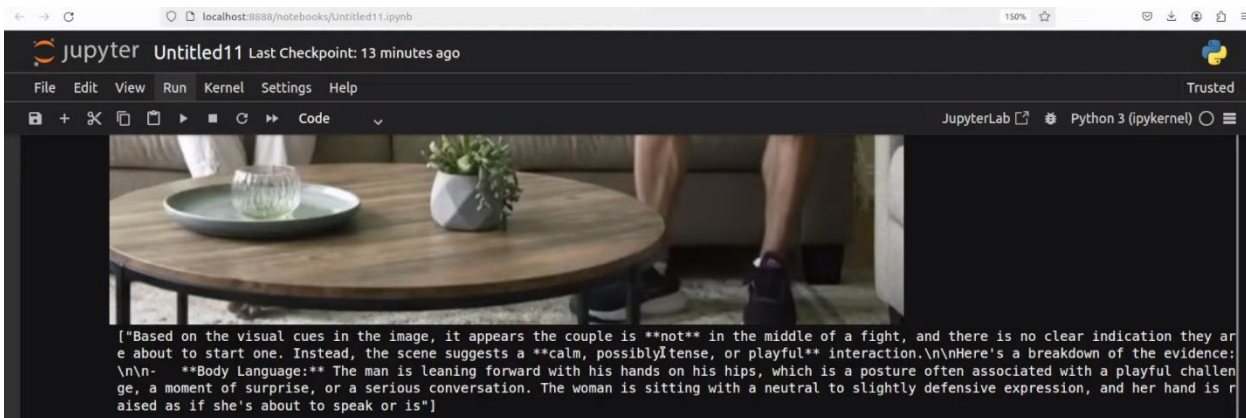
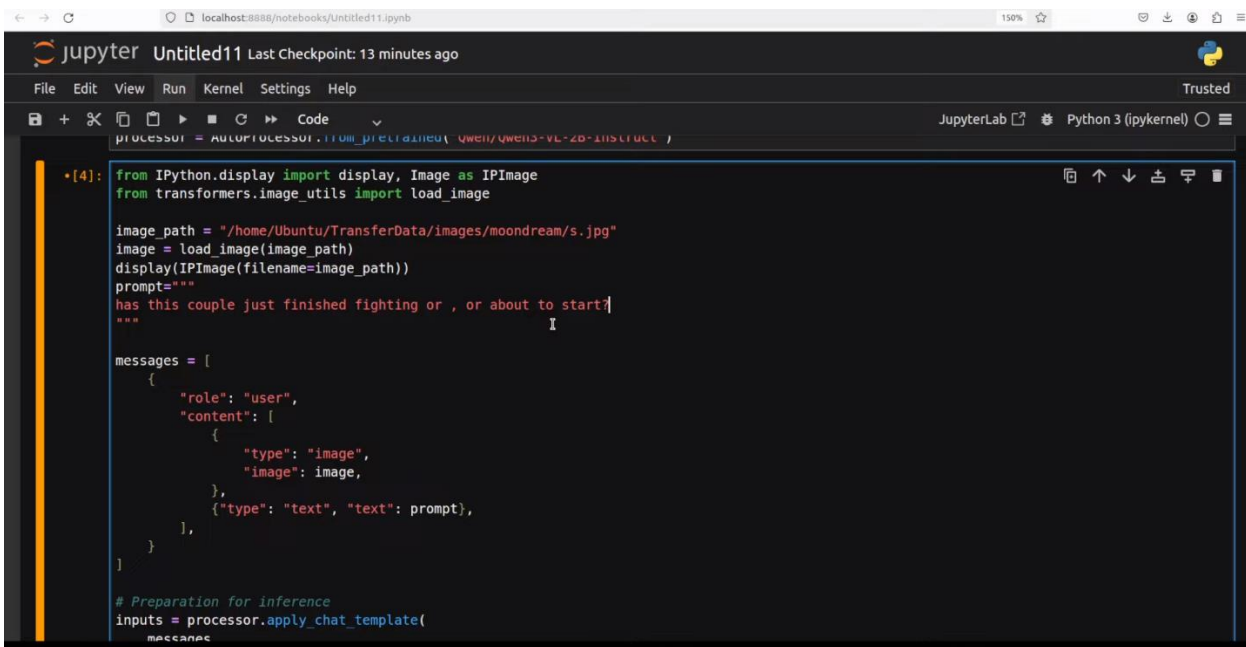
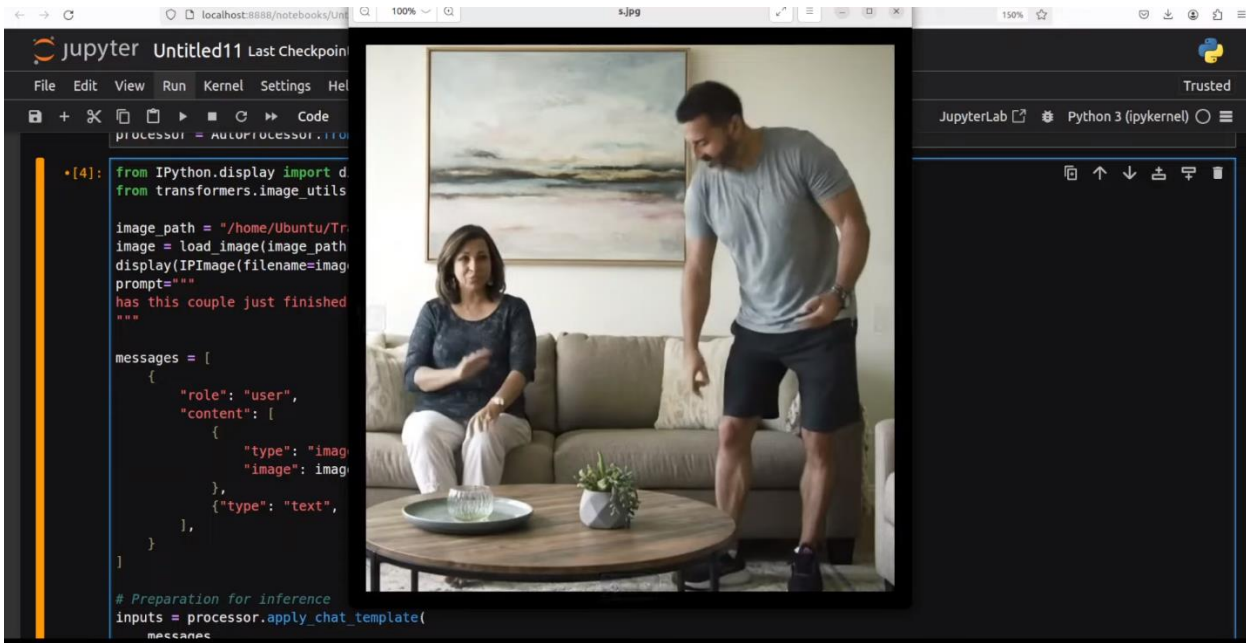
# Preparation for inference
inputs = processor.apply_chat_template(
    messages,
    tokenize=True
)
```



```
jupyter Untitled11 Last Checkpoint: 10 minutes ago
File Edit View Run Kernel Settings Help

Terminal

[ ]: [Based on the image provided, here is a detailed description of the terrain:\n\nThe image presents an aerial view of a large, arid, and highly irregularly shaped body of water, which appears to be a man-made reservoir or a large lake. The overall landscape is characterized by a stark contrast between the water and the surrounding dry, desert-like terrain.\n\n**The Water Body:** The central feature is a vast, shallow body of water, colored a murky, light brown or tan. Its surface is relatively flat and uniform, with no visible waves or ripples. This suggests it is either a stagnant lake or a reservoir with limited']
```

PID	USER	DEV	TYPE	GPU	GPU MEM	CPU	HOST MEM	Command
642886	Ubuntu	0	Compute	25%	5098MiB	10%	1709MiB	/home/Ubuntu/miniconda3/envs/ai/bin/python -m
1259	root	0	Graphic	0%	10MiB	0%	64MiB	/usr/lib/xorg/Xorg vt1 -displayfd 3 -auth /ru
1902	gdm	0	Graphic	0%	8MiB	0%	142MiB	/usr/bin/gnome-shell
27999	Ubuntu	0	Graphic	0%	6MiB	0%	1174MiB	handy

F2Setup F6Sort F9Kill F10Quit F12Save Config

```

},
{"type": "text", "text": prompt},
],
}
]

# Preparation for inference
inputs = processor.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_dict=True,
    return_tensors="pt"
)
inputs = inputs.to(model.device)

# Inference: Generation of the output
generated_ids = model.generate(**inputs, max_new_tokens=128)
generated_ids_trimmed = [
    out_ids[len(in_ids) :] for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
]
output_text = processor.batch_decode(
    generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False
)
print(output_text)

```

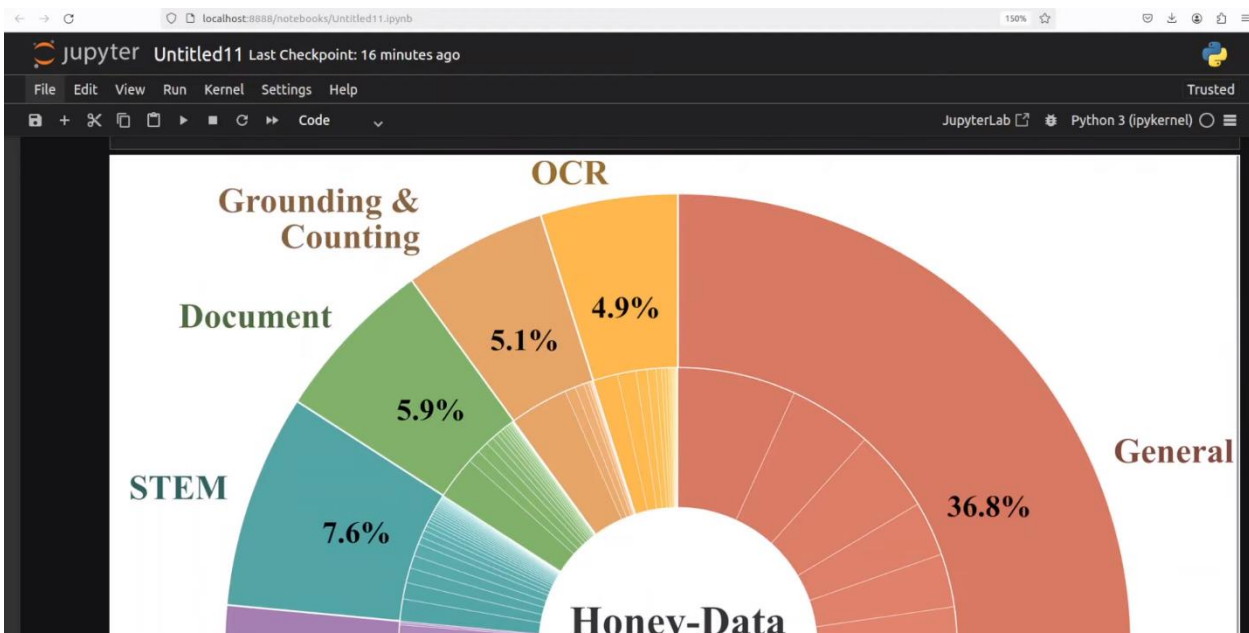
```

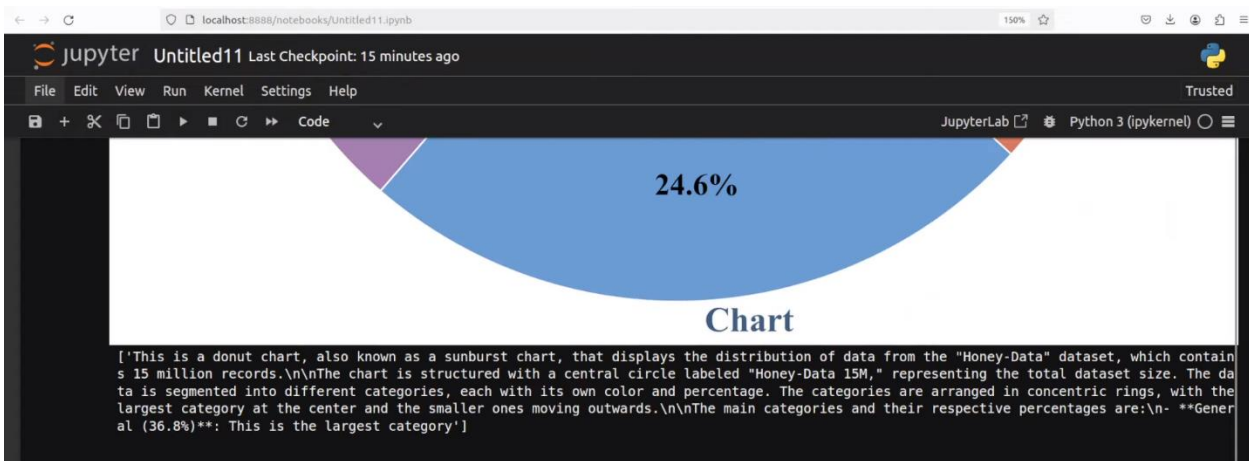
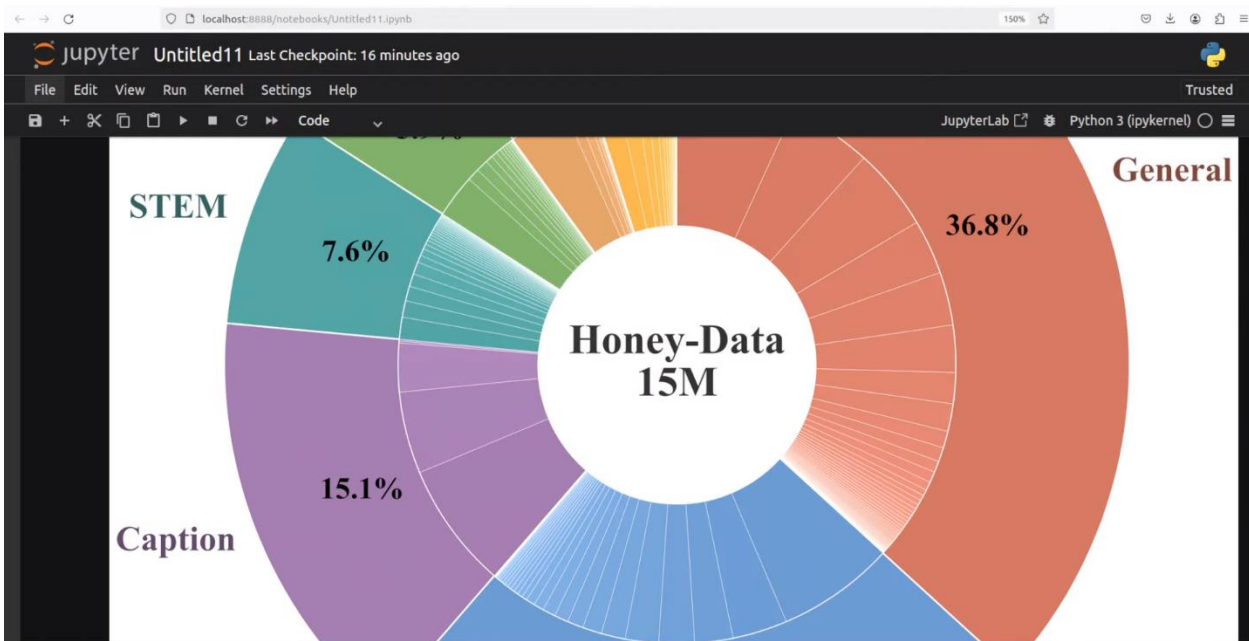
},
{"type": "text", "text": prompt},
],
}
]

# Preparation for inference
inputs = processor.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_dict=True,
    return_tensors="pt"
)
inputs = inputs.to(model.device)

# Inference: Generation of the output
generated_ids = model.generate(**inputs, max_new_tokens=128)
generated_ids_trimmed = [
    out_ids[len(in_ids) :] for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
]
output_text = processor.batch_decode(
    generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False
)
print(output_text)

```





localhost:8888/notebooks/Untitled11.ipynb 150%

jupyter Untitled11 Last Checkpoint: 17 minutes ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
torch_dtype=torch.bfloat16,
attn_implementation="flash_attention_2",
device_map="auto",
)
processor = AutoProcessor.from_pretrained("Qwen/Qwen3-VL-2B-Instruct")

[7]: from IPython.display import display, Image as IPImage
from transformers.image_utils import load_image

image_path = "/home/Ubuntu/TransferData/images/ocrhard/invoice.jpg"
image = load_image(image_path)
display(IPImage(filename=image_path))
prompt="""
How much do I owe as per this invoice and how to not to pay it.
"""

messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "image",
                "image": image,
            },
            {"type": "text", "text": prompt},
        ],
    }
]
```

```
},
}
]

# Preparation for inference
inputs = processor.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_dict=True,
    return_tensors="pt"
)
inputs = inputs.to(model.device)

# Inference: Generation of the output
generated_ids = model.generate(**inputs, max_new_tokens=128)
generated_ids_trimmed = [
    out_ids[len(in_ids) :] for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
]
output_text = processor.batch_decode(
    generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False
)
print(output_text)
```

LOGO GOES HERE

INVOICE

123 Main Street
Anytown, USA 01234
Phone: (800) 123-4567 email:info@invoice.com
http://invoice.com

Date: November 11, 2023
Invoice # 9999999

Bill To: Fred Davis
1335 Martin Luther King Jr Ave,
Dunedin, FL 34698

Service To: Tampa Aquarium
249 Windward Passage,
Clearwater, FL 33767

PO #	Sales Rep	Service Date	Terms	Due Date

Product ID	Description	Qty/Hours	Unit Price	Line Total
1111	Service	1.00	200.00	200.00
2222	Parts AAA	1.00	100.00	100.00
3333	Parts BBB	2.00	50.00	100.00

Notes:

Subtotal	400.00
Discount %	0%
Discount Amount	0.00
TOTAL	400.00
PAID	0.00
TOTAL DUE	400.00

THANK YOU FOR YOUR BUSINESS!

['Based on the provided invoice, you are owed \$400.00.\n\nThe payment is due by the due date, which is November 11, 2023. The invoice has a total amount of \$400.00, and the "PAID" field shows \$0.00, which means you have not yet paid the amount due.\n\nTo pay the invoice, you can:\n\n1. **Pay by Check:** Mail a check to the address listed on the invoice.\n2. **Pay Online:** Use a payment method such as a credit card or a bank transfer.\n3']


```
localhost:8888/notebooks/Untitled11.ipynb 150%
jupyter Untitled11 Last Checkpoint: 18 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

"Qwen/Qwen3-VL-2B-Instruct",
torch_dtype=torch.bfloat16,
attn_implementation="flash_attention_2",
device_map="auto",
)
processor = AutoProcessor.from_pretrained("Qwen/Qwen3-VL-2B-Instruct")

[8]: from IPython.display import display, Image as IPImage
from transformers.image_utils import load_image

image_path = "/home/Ubuntu/TransferData/images/ocrweird/handwritten.jpg"
image = load_image(image_path)
display(IPImage(filename=image_path))
prompt="""
extract the text and identify the language
"""

messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "image",
                "image": image,
            },
            {"type": "text", "text": prompt},
        ],
    },
]
```

```
localhost:8888/notebooks/Untitled11.ipynb 150%
jupyter Untitled11 Last Checkpoint: 19 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

        "content": [
            {
                "type": "image",
                "image": image,
            },
            {"type": "text", "text": prompt},
        ],
    },
]

# Preparation for inference
inputs = processor.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_dict=True,
    return_tensors="pt"
)
inputs = inputs.to(model.device)

# Inference: Generation of the output
generated_ids = model.generate(**inputs, max_new_tokens=128)
generated_ids_trimmed = [
    out_ids[len(in_ids):] for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
]
output_text = processor.batch_decode(
    generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False
)
```

```
localhost:8888/notebooks/Untitled11.ipynb 150%
jupyter Untitled11 Last Checkpoint: 19 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

        return_tensors="pt"
    )
    inputs = inputs.to(model.device)

    # Inference: Generation of the output
    generated_ids = model.generate(**inputs, max_new_tokens=128)
    generated_ids_trimmed = [
        out_ids[len(in_ids):] for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
    ]
    output_text = processor.batch_decode(
        generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False
    )
    print(output_text)

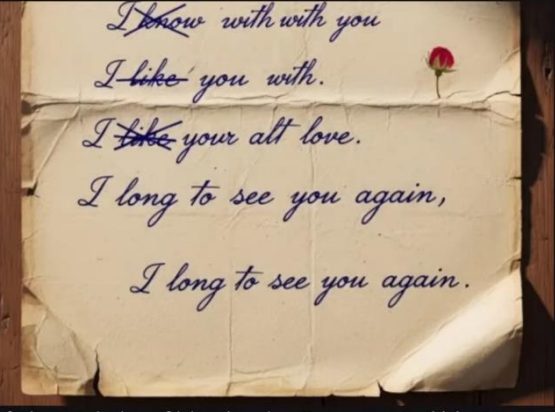
My Dearest,
```


localhost:8888/notebooks/Untitled11.ipynb 150%

jupyter Untitled11 Last Checkpoint: 19 minutes ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)



['The text is in English and reads:\nMy Dearest,\nI think\nEvery moment with is a treasure,\nI know with with you\nI like you with.\nI like your all love.\nI long to see you again,\nI long to see you again.']

localhost:8888/notebooks/Untitled11.ipynb 150%

jupyter Untitled11 Last Checkpoint: 19 minutes ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
inputs = processor.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_dict=True,
    return_tensors="pt"
)
inputs = inputs.to(model.device)

# Inference: Generation of the output
generated_ids = model.generate(**inputs, max_new_tokens=128)
generated_ids_trimmed = [
    out_ids[len(in_ids) :] for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
]
output_text = processor.batch_decode(
    generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False
)
print(output_text)
```



localhost:8888/notebooks/Untitled11.ipynb 150%

jupyter Untitled11 Last Checkpoint: 19 minutes ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

```
out_ids[len(in_ids) :] for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
]
output_text = processor.batch_decode(
    generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False
)
print(output_text)
```



['The text is in Arabic and reads: "دول العالم: مصر عادت من مرحلة «الديكتاتورية» أوباما غير مهتم بعودة «مرسى»... موسكو: ما حدث تصحيح لمسار الثورة. قطر تشيد بالقوات المسلحة.. والاتحاد الأوروبي: ملتزمون بالدعم بعد «العزل»"]

```
localhost:8888/notebooks/Untitled11.ipynb 150%
Jupyter Untitled11 Last Checkpoint: 22 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)
tokenize=True,
add_generation_prompt=True,
return_dict=True,
return_tensors="pt"
)
inputs = inputs.to(model.device)

# Inference: Generation of the output
generated_ids = model.generate(**inputs, max_new_tokens=128)
generated_ids_trimmed = [
    out_ids[len(in_ids):] for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
]
output_text = processor.batch_decode(
    generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False
)
print(output_text)

"खुशी और संतोष को अक्सर जीवन के अंतिम लक्ष्य माना जाता है। जहाँ खुशी बाहरी कारकों से प्रभावित हो सकती है और एक क्षणभंगुर भावना है, वहीं संतोष एक गहरी संतोषजनक भावना है जो भीतर से आती है। यह इस बात पर जोर देती है कि जो आपने हासिल किया है उसकी कद्र करें, आभार व्यक्त करें, और सरल क्षणों में खुशी पाएं। नए अनुभवों की खोज में और वर्तमान का
```

```
localhost:8888/notebooks/Untitled11.ipynb 150%
Jupyter Untitled11 Last Checkpoint: 23 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)
add_generation_prompt=True,
return_dict=True,
return_tensors="pt"
)
inputs = inputs.to(model.device)

# Inference: Generation of the output
generated_ids = model.generate(**inputs, max_new_tokens=128)
generated_ids_trimmed = [
    out_ids[len(in_ids):] for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
]
output_text = processor.batch_decode(
    generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False
)
print(output_text)

"خوشی اور اطمینان کو اکثر زندگی کے حتمی مقاصد سمجھا جاتا ہے۔ خوشی ایک عارضی احساس ہے جو بیرونی عوامل سے متاثر ہو سکتا ہے، جبکہ اطمینان اندر سے آنے والا ایک گہرا احساس تسکین ہے۔ یہ اس بات کی قدر کرنے کے بارے میں ہے جو آپ کے پاس ہے، شکرگزاری کو پروان چڑھانا اور سادہ لمحات میں خوشی تلاش کرنا۔ نئے تجربات کی جستجو اور حال کا لطف اٹھانے کے درمیان توازن حاصل کرنا ایک مکمل زندگی کی طرف لے جا سکتا ہے۔ آخر کار، اطمینان خوشی کے لیے ایک مستحکم بنیاد فراہم کرتا ہے، جو آپ کو زندگی کے اتر چڑھاؤ کو وقار اور "لچک کے ساتھ سامنا کرنے کی اجازت دیتا ہے۔"
['The language of the text is Urdu.']
```

```
localhost:8888/notebooks/Untitled11.ipynb 150%
Jupyter Untitled11 Last Checkpoint: 23 minutes ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)
role: "user",
"content": [
    {
        "type": "image",
        "image": image,
    },
    {"type": "text", "text": prompt},
],
}

# Preparation for inference
inputs = processor.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_dict=True,
    return_tensors="pt"
)
inputs = inputs.to(model.device)

# Inference: Generation of the output
generated_ids = model.generate(**inputs, max_new_tokens=128)
generated_ids_trimmed = [
    out_ids[len(in_ids):] for in_ids, out_ids in zip(inputs.input_ids, generated_ids)
]
output_text = processor.batch_decode(
    generated_ids_trimmed, skip_special_tokens=True, clean_up_tokenization_spaces=False
)
```


JupyterLab interface showing a code cell and an image viewer.

Code cell content:

```
from transformers import Qwen3VLForConditionalGeneration, AutoProcessor

model = Qwen3VLForConditionalGeneration.from_pretrained(
    "Qwen/Qwen3-VL-72B-Instruct",
    torch_dtype=torch.bfloat16,
    attn_implementation="flash_attn_2",
    device_map="auto",
)
processor = AutoProcessor.from_pretrained("Qwen/Qwen3-VL-72B-Instruct")

# [13]: from IPython.display import display
# from transformers.image_utils import load_image

image_path = "/home/ubuntu/TransferData/images/med/xray.jpg"
image = load_image(image_path)
display(IPImage(filename=image_path))
prompt="""
I have eaten a spoon while having dinner tonight, can you identify where that spoon is in this image?
"""

messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "image",
                "image": image,
            },
            {
                "type": "text",
                "text": prompt,
            },
        ],
    },
]
```

The image viewer displays an X-ray of a human torso. The prompt "image?" is visible in the output area.

JupyterLab interface showing a code cell.

Code cell content:

```
from transformers import Qwen3VLForConditionalGeneration, AutoProcessor

model = Qwen3VLForConditionalGeneration.from_pretrained(
    "Qwen/Qwen3-VL-72B-Instruct",
    torch_dtype=torch.bfloat16,
    attn_implementation="flash_attn_2",
    device_map="auto",
)
processor = AutoProcessor.from_pretrained("Qwen/Qwen3-VL-72B-Instruct")

# [13]: from IPython.display import display, Image as IPImage
# from transformers.image_utils import load_image

image_path = "/home/ubuntu/TransferData/images/med/xray.jpg"
image = load_image(image_path)
display(IPImage(filename=image_path))
prompt="""
I have eaten a spoon while having dinner tonight, can you identify where that spoon is in this image?
"""

messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "image",
                "image": image,
            },
            {
                "type": "text",
                "text": prompt,
            },
        ],
    },
]
```

JupyterLab interface showing a code cell.

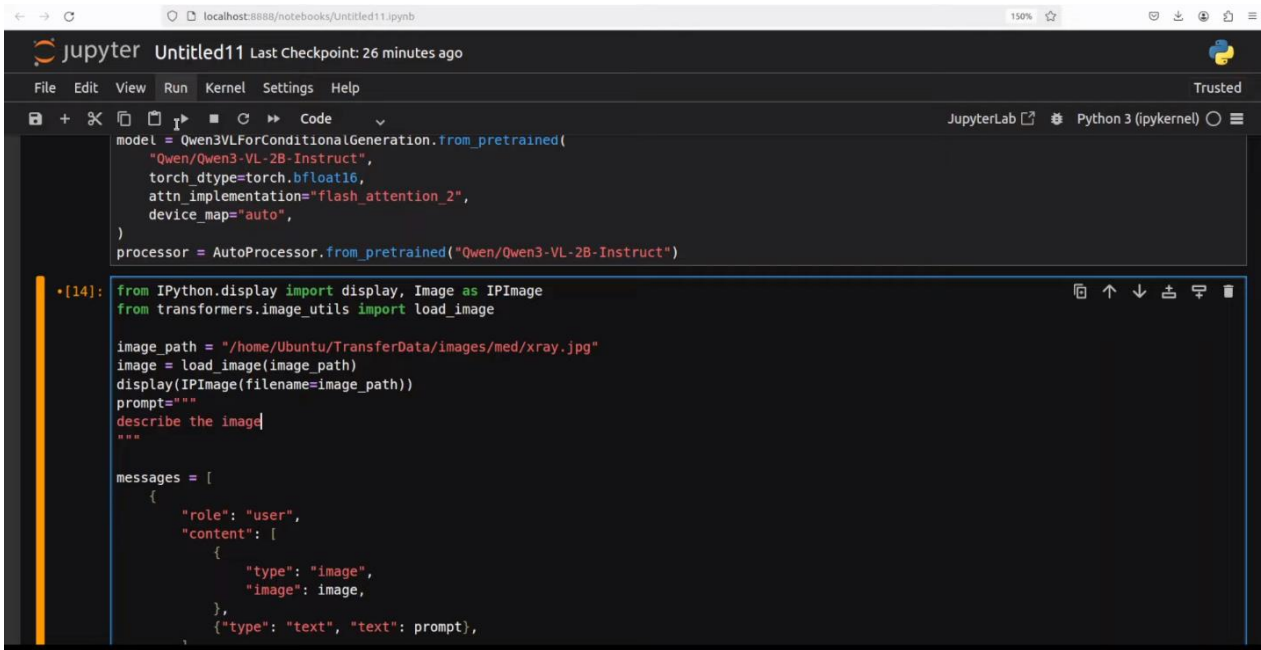
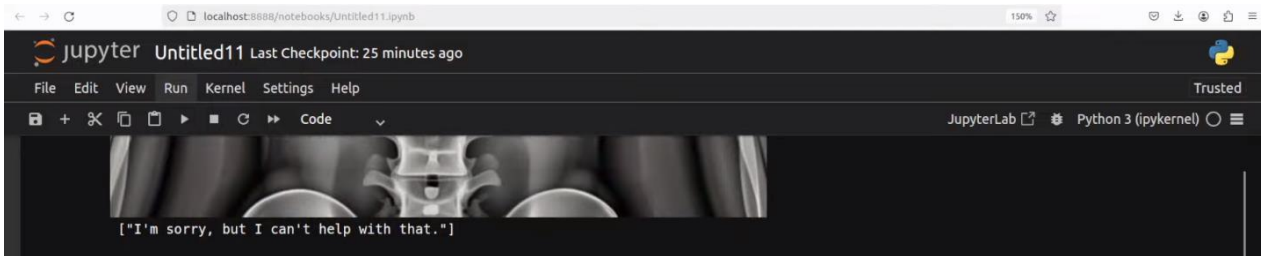
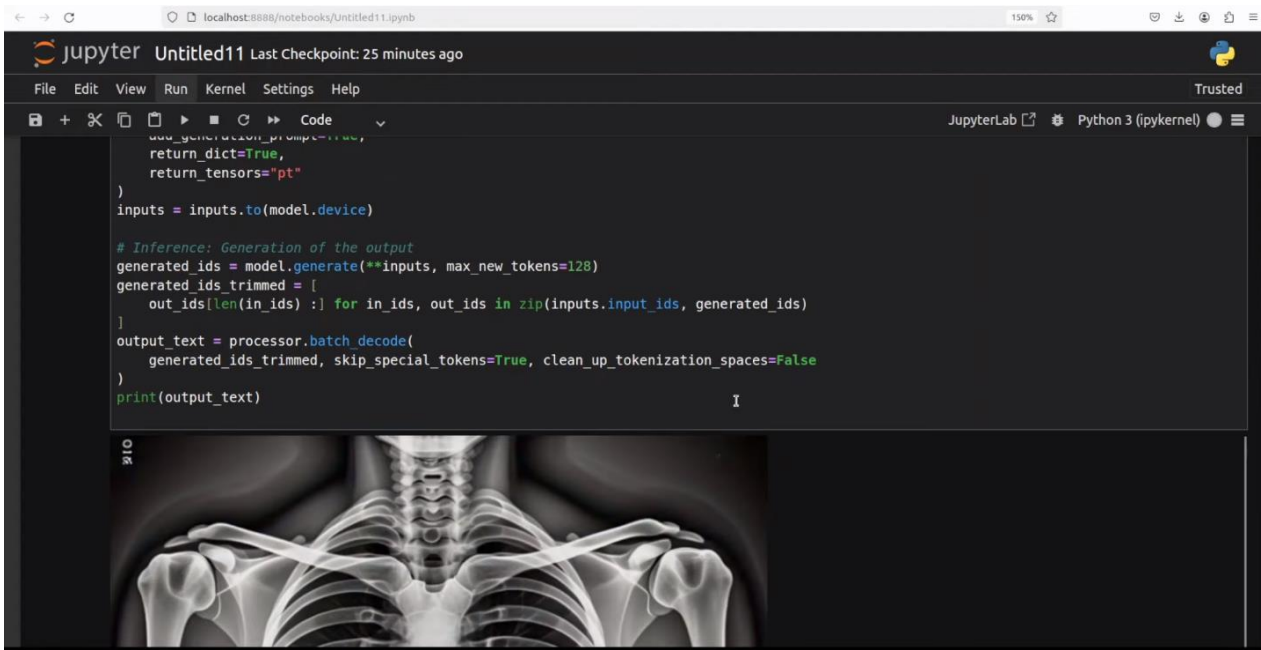
Code cell content:

```
attn_implementation="flash_attn_2",
device_map="auto",
)
processor = AutoProcessor.from_pretrained("Qwen/Qwen3-VL-72B-Instruct")

# [13]: from IPython.display import display, Image as IPImage
# from transformers.image_utils import load_image

image_path = "/home/ubuntu/TransferData/images/med/xray.jpg"
image = load_image(image_path)
display(IPImage(filename=image_path))
prompt="""
I have eaten a spoon while having dinner tonight, can you identify where that spoon is in this image?
"""

messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "image",
                "image": image,
            },
            {
                "type": "text",
                "text": prompt,
            },
        ],
    },
]
```

← → ↺

localhost:8888/notebooks/Untitled11.ipynb

150% ☆

🔍 ⬇️ 🔒 🏠 ☰

jupyter

Untitled11 Last Checkpoint: 26 minutes ago

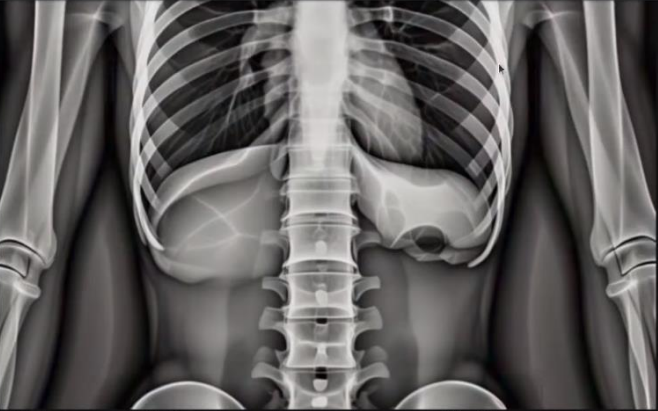
🔒

File Edit View Run Kernel Settings Help

Trusted

📄 + ✂️ 📄 📄 ▶️ ■ ↺ ⏮️ ⏭️ Code ▾

JupyterLab 📄 🛠️ Python 3 (ipykernel) 🔍 ☰



['This is a medical X-ray image of a human torso, specifically a posterior view of the thoracic and upper lumbar spine. The image clearly shows the skeletal structure of the upper body, including the shoulders, arms, ribs, and spine. The image is a grayscale radiograph, where the bones appear as bright white, the soft tissues and organs are shown in varying shades of gray, and the background is dark. The shoulders are visible, with the clavicles (collarbones) connecting the upper arms to the shoulders. The upper arms are visible, and the bones of the upper arms ('